詳細設計書
ライントレースカー
組込みソフトウェア

1. 文書の目的

本書は、ライントレースカーの組込みソフトウェアにおける各モジュールの詳細設計を記述し、コーディングの指針とすることを目的とする。

2. モジュール設計詳細

本システムのモジュールは以下の機能で構成される：

1_メイン制御モジュール(計算制御含む)

2_入力制御モジュール

3_出力制御モジュール

4_表示制御モジュール

5_音制御モジュール

3. 関数仕様

2_入力制御モジュール

関数名：task_2_sensor_main

プロトタイプ：int task_5_sound_main (void)

引数：なし(void)

戻り値：int

処理内容：

入力制御モジュールにおけるメイン関数。入力制御（センサ）の信号をメイン制御に送る。更新されたシステム状態(gsm_SysState)に応じて各関数を呼び出す。

関数名：pinMode

プロトタイプ：なし（WiringPi.h に格納）

引数：

戻り値：なし(void)

処理内容：

入力制御において、距離センサ、フォトセンサ、赤・白スイッチを PWM_INPUT、PUD_DOWN に設定。入力を制御する degitalRead 関数を有効にする。

関数名：ReadSens

プロトタイプ：int ReadSens(void)

引数：int

戻り値：なし(void)

処理内容：

フォトセンサがラインかライン外かを検知した際、２進数に変換するように処理。

int で２進数の値を送る。赤、緑センサはライン内の際は０を、黄センサは１を送る。

関数名： ReadDistance

プロトタイプ：int ReadDistance(int fd2)

引数：int fd2

戻り値：int

処理内容：

距離センサで得た値をビットでｃｍに直す処理を行う関数。fd2 は現在、距離センサで得たｃｍに直していない値が入る。

3_出力制御モジュール

関数名： task_3_output_main 関数

プロトタイプ： int task_3_output_main(void)

引数： なし（void）

戻り値： 正常終了（EXIT_SUCCESS）

処理内容： システム状態と駆動指示値に応じて、モーターの初期化、前進・後退・旋回・停止制御を行う。

関数名： Stop 関数

プロトタイプ： void Stop(void)

引数： なし（void）

戻り値： なし（void）

処理内容： 左右のモーター PWM 値をすべて 100 に設定し、停止制御を行う。

関数名： LittleForward 関数

プロトタイプ： void LittleForward(void)

引数： なし（void）

戻り値： なし（void）

処理内容： 左右のモーターを正転させ、右側 PWM 値 30、左側 PWM 値 30 で低速前進する。サーボ PWM 値を 14 に設定する。

関数名： Forward 関数

プロトタイプ： void Forward(void)

引数： なし（void）

戻り値： なし（void）

処理内容： 左右のモーターを正転させ、右側 PWM 値 100、左側 PWM 値 100 で前進する。サーボ PWM 値を 14 に設定する。

関数名： LittleRightForward 関数

プロトタイプ： void LittleRightForward(void)

引数： なし（void）

戻り値： なし（void）

処理内容： 左右のモーターを正転させ、右側 PWM 値 40、左側 PWM 値 100 で緩やかに右折する。サーボ PWM 値を 10 に設定する。

関数名： RightForward 関数

プロトタイプ： void RightForward(void)

引数： なし（void）

戻り値： なし（void）

処理内容： 左右のモーターを正転させ、右側 PWM 値 20、左側 PWM 値 100 で大きく右折する。サーボ PWM 値を 10 に設定する。

関数名： LittleLeftForward 関数

プロトタイプ： void LittleLeftForward(void)

引数： なし（void）

戻り値： なし（void）

処理内容： 左右のモーターを正転させ、右側 PWM 値 100、左側 PWM 値 40 で緩やかに左折する。サーボ PWM 値を 18 に設定する。

関数名： LeftForward 関数

プロトタイプ： void LeftForward(void)

引数： なし（void）

戻り値： なし（void）

処理内容： 左右のモーターを正転させ、右側 PWM 値 100、左側 PWM 値 20 で大きく左折する。サーボ PWM 値を 18 に設定する。

関数名： Reverse 関数

プロトタイプ： void Reverse(void)

引数： なし（void）

戻り値： なし（void）

処理内容： 左右のモーターを逆転させ、右側 PWM 値 30、左側 PWM 値 30 で低速後退する。サーボ PWM 値を 14 に設定する。

関数名： SlowStop 関数

プロトタイプ： void SlowStop(void)

引数： なし（void）

戻り値： なし（void）

処理内容： 左右のモーター PWM 値を 60 から 100 まで 10 ずつ段階的に増加させ停止処理を行う。

関数名： ONESTOP_SlowStop 関数

プロトタイプ： void ONESTOP_SlowStop(void)

引数： なし（void）

戻り値： なし（void）

処理内容： 障害物検知時に、左右のモーター PWM 値を 60 から 100 まで 10 ずつ段階的に増加させ停止処理を行う。

4 表示制御モジュール

関数名：task_4_lcdled_main

プロトタイプ： int task_4_lcdled_main(void)

引数： なし（void）

戻り値： 正常終了（EXIT_SUCCESS）

処理内容： システム状態に応じて、LCD モニターの文字と LED を変化させる

関数名：LcdClear

プロトタイプ：int LcdClear(int fd)

引数：fd　 wiringPiI2CSetup 関数で取得したファイルハンドル

戻り値: エラーの時は-1

処理内容：LCD の画面に表示されている内容を消去する。

関数名：LcdSetup

プロトタイプ：int LcdSetup(int fd)

引数：fd

戻り値：エラーのときは-1

処理内容：LCD の初期化

関数名：LcdWriteString

プロトタイプ：int LcdWriteString(int fd, char *s)

引数：fd ,*s　文字列

戻り値：エラーのときは-1

処理内容：LCD に文字列を表示

5_音制御モジュール

関数名：task_5_sound_main

プロトタイプ：int task_5_sound_main (void)

引数：なし(void)

戻り値：int

処理内容：音制御モジュールのなかにおけるメイン関数。更新されたシステム状態(gsm_SysState)に応じて各関数を呼び出す。

関数名：PlayMusic

プロトタイプ：void PlayMusic(void)

引数：なし(void)

戻り値：なし(void)

処理内容：SM_RUNNING へ移行後、音楽を再生する。本件では、曲名「さくら」の冒頭の音符一つずつを順に格納する配列を用意して、それを再生することとする。

関数名：PlayObstacleBuzzer

プロトタイプ：int PlayObstacleBuzzer(void)

引数：なし(void)

戻り値：int

処理内容：障害物を検知して SM_ONESTOP へ移行後、ブザーを鳴らす。

関数名：g

プロトタイプ：void g (int oct, int solfa, int length);

引数：

oct：3:下のオクターブ、4:中央、5:上のオクターブ

solfa：0:休符、1:ド、2:ド#、3:レ、4:レ#、5:ミ、6:ファ、7:ファ#、8:ソ、9:ソ#、10:ラ、11:ラ#、12:シ

length：時間の長さ 単位 ms

戻り値：なし(void)

処理内容：配列を別に用意して、引数で指定された音名または休符を一つずつハードウェア PWM で再生する。各音階の周波数を求める式は、440×(2 の 12 乗根)の d 乗とする。変数 d は指数であり、ドからシにかけて順次、整数の-9 から 2 とする。

を発生する。Oct はオクターブ、solfa は階名。#define で CLK は 19200000、pwmRange の引数 RANGE を 100 と定義すること。

4. データ構造

```
typedef enum _SM_MODE
{
        SM_INIT = 0,
        SM_WAIT,
        SM_RUNNING,
        SM_ONESTOP,
        SM_EMER,
} SM_MODE;
```

5. エラー処理

各関数は入力値の妥当性を確認し、障害物を検知した場合は、一時停止をしてブザーを鳴らすことで、ユーザーに通知する。

6. インターフェース定義

各機能間は関数呼び出しにより連携する。外部 I/O は GPIO 経由でボタン入力と LED・LCD の制御を行う。モーター・圧電サウンダ制御は PWM を使用して制御する。